

Maintaining High-Context Awareness in an AI-Assisted Project

Developing a complex web/mobile app with multiple services requires careful documentation of context so that both developers and AI assistants (like Cursor) can stay informed. Below is a structured plan to create **standardized context files** at the project root and within each service. This will help maintain persistent, navigable, and auto-updating technical and project context throughout development.

Project Structure and Context Files

Organize your repository to include a set of context files at the root (for global context) and within each service (for service-specific context). The table below outlines a recommended structure and naming conventions, along with the purpose of each file:

Root-Level Context Files (Project-Wide)

File Name	Purpose and Contents
README.md	Project Overview: High-level description of the project, its goals, and its overall architecture. Include the purpose of the application, key features, and a summary of how the major components fit together. This helps new contributors and AI understand the project's mission and scope at a glance.
CONTEXT.md	Detailed Project Context & Architecture: Expanded context that doesn't fit in the brief README. This can include a system architecture overview (e.g. diagrams or text describing how frontend, backend, and worker services interact), domain knowledge, and any background information. It provides deeper technical context and rationale behind the architecture. Consider linking to service-specific context files for more details on each component.
MANAGER.md	Project Management & Decisions Log: A running log of technical decisions, design choices, and trade-offs made across the project. Each entry might include the date, decision summary, options considered, and rationale. This file serves as an internal "project manager," recording why certain frameworks or patterns were chosen (and alternatives rejected) and other historical notes. It can also include high-level planning or roadmap items. By capturing decisions and their context here, you create a persistent memory of <i>why</i> the codebase is the way it is.

File Name	Purpose and Contents
TRACKER.md	Tasks & Blockers Tracker: A living to-do list and progress tracker for the project. Use this to list planned features, tasks in progress, completed tasks, and any current blockers or issues. A simple format is a bullet list or checklist (e.g. using <code>- []</code> Markdown checkboxes for open tasks and <code>- [x]</code> for done). This gives both developers and AI a quick view of the project's status and what's next. Historical sections can be added to log past sprints or releases (or you might maintain a separate <code>CHANGELOG.md</code> for released versions).
INTERFACE.md	Inter-Service Relationships: Documentation of how all the services interact. List each service and its interfaces with others – for example, API endpoints exposed (and which service uses them), message queue topics or events shared between the API and worker, databases or caches shared, etc. A table or clear list can map out connections (e.g. "API service → Worker service: sends tasks to <code>jobs_queue</code>" or "Mobile app → API: uses REST endpoints <code>/api/v1/...</code>"). This global interface catalog ensures nothing falls through the cracks when reasoning about cross-service behavior.

Service-Level Context Files (within each service folder)

For each service (e.g. `api/`, `mobile/`, `worker/`), include a similar set of files capturing context specific to that component:

File Name	Purpose and Contents (Service-Specific)
README.md	Service Overview: Purpose of this service, how to set it up and run it, and its role in the overall system. This might include its main functionalities and which other components or users interact with it. (For example, the API service README would note it's a backend providing REST endpoints for the mobile app and sending jobs to the worker.)
CONTEXT.md	Service Technical Context: Important technical details and architecture of this service. Describe the service's internal structure (key modules, frameworks used, design patterns) and any service-local decisions or conventions. For instance, if the API uses a certain caching strategy or the mobile app uses a specific state management library, note it here. This file gives the AI and developers deeper insight into the inner workings of the service, without having to dive into code immediately.
INTERFACE.md	Service Interface Details: Focused description of how <i>this</i> service connects to others. Document the service's API (endpoints or public functions) and any external APIs or queues it consumes. Essentially, it's a subset of the global INTERFACE.md but scoped to this service: e.g., "This worker subscribes to <code>jobs_queue</code> from the API" or "Mobile app calls these endpoints on the API, and listens for these push notification events." Keeping this info in the service folder makes it easy to update when you change the service's API.

File Name	Purpose and Contents (Service-Specific)
TRACKER.md	Service Task Tracker: A to-do list specific to this service. List features or fixes to implement in the service, progress, and any blockers that specifically affect this component. This can be more granular than the project-wide tracker (for example, "Implement payment API integration [blocked by obtaining API keys]"). Developers working in this service can refer here for pending work. If a task involves multiple services, note it in the project-wide TRACKER.md as well, but for single-service tasks, keep them here.
(Optional) DECISIONS.md / HISTORY.md	Service Decision Log: If the service has significant design decisions on its own (e.g. choice of database, or a refactor history), you can maintain a log here (similar to MANAGER.md but scoped to this service). This is optional if MANAGER.md (global) already covers most decisions, but in a large project it can help to separate global vs. service-specific decisions. Architecture Decision Record (ADR) style entries could be used here.

File Naming Conventions: The names above (README, CONTEXT, etc.) are capitalized for clarity and to stand out. Using `.md` Markdown format is ideal for readability. You can adapt the names (e.g. `DECISIONS.md` instead of `MANAGER`) but keep names self-explanatory so developers and AI can easily infer their contents. For example, having "INTERFACE" in the file name makes it clear it contains interface/integration info, which an AI may retrieve when asked about service communication.

Directory Layout Example:

```

project-root/
├── README.md           <- Project goals, overview, high-level architecture
├── CONTEXT.md          <- Detailed context and architecture info
├── MANAGER.md          <- Decisions and trade-offs log (project-wide)
├── TRACKER.md          <- Global tasks and blockers
├── INTERFACE.md        <- All inter-service relationships
├── api/                <- API service directory
│   ├── README.md       <- API service overview and usage
│   └── CONTEXT.md      <- API-specific technical context (design,
patterns)
│   ├── INTERFACE.md    <- API interfaces (endpoints, outgoing messages)
│   ├── TRACKER.md      <- API-specific tasks & issues
│   └── ... (code) ...
├── mobile/             <- Mobile app service directory
│   ├── README.md       <- Mobile app overview
│   └── CONTEXT.md      <- Mobile-specific context (architecture,
libraries)
│   ├── INTERFACE.md    <- Mobile interfaces (API calls made, etc.)
│   ├── TRACKER.md      <- Mobile-specific tasks & backlog
│   └── ... (code) ...
└── worker/             <- Worker service directory

```

— README.md	<- Worker service overview
— CONTEXT.md	<- Worker-specific context (purpose, internals)
— INTERFACE.md	<- Worker interfaces (queues/topics, APIs used)
— TRACKER.md	<- Worker-specific tasks & backlog
— ... (code) ...	

This structure ensures that for every layer (project-wide and service-level), there are dedicated places to look for context. It's similar in spirit to emerging conventions like the Codebase Context Specification, which suggests keeping a structured context directory (e.g. a `.context/` folder with an index and docs) for AI agents ¹. The key is that context is documented *where it's most relevant* – global info at the root, and service info in that service's folder.

Capturing Key Project Information

When populating these files, focus on capturing the following core types of information:

- **Project Goals & High-Level Architecture:** In the root **README.md**, clearly state the project's purpose, target users, and primary features. Summarize the high-level architecture (perhaps with a simple diagram or list of components) so readers know the big picture. If the architecture is complex, use **CONTEXT.md** or an **ARCHITECTURE.md** (which could be your CONTEXT file) to describe it in depth – e.g. explaining the web/mobile/client-server setup, any microservices, and how data flows. This sets the stage for any AI assistance; the AI can be directed to these files to understand the system's design before generating code.
- **Technical Decisions and Trade-offs:** Every time a significant decision is made (choice of framework, library, architectural pattern, etc.), record it in **MANAGER.md** or a **DECISIONS.md**. For each decision, note what options were considered, which was chosen, and why. For example: **"Decision (2025-05-21):** Chose PostgreSQL over MongoDB for user data – rationale: need for complex joins and transactions; trade-off was sacrificing some flexibility in schema ²." By logging decisions and their dates, you create a timeline of rationale. This helps future developers (or an AI) understand context, so they don't unknowingly reverse a past decision or re-debate settled questions. If a decision pertains only to one service, log it in that service's context or decisions file; if it's broad (affecting multiple components), put it in the project-wide log. Consider using a simple template (bullet points or a table) for each entry (Date, Decision, Alternatives, Reason) to keep it consistent. This practice is akin to maintaining Architecture Decision Records, but can be done informally in the Markdown files.
- **Inter-Service Relationships:** Use **INTERFACE.md** (both globally and per service) to capture how components communicate. It's helpful to list interactions in a structured way. For example, you might create a table of API endpoints (endpoint, purpose, who calls it) in the API service's interface file, and a table of message queue topics (topic name, publisher, consumer) in the global interface file or relevant service files. The goal is to make integration points explicit. If the AI needs to modify how two services interact, it can refer to these docs to see the contract between services. Likewise, if you have shared schemas or data models, note which service owns the source of truth. Keeping this updated means whenever you add or change an interface (say, add a new API endpoint or queue message), you immediately update the doc in that service (and possibly a summary in the root

INTERFACE.md). This is invaluable for context: an AI can be pointed to “@api/INTERFACE.md” to understand all endpoints of the API, for instance.

- **Tasks, Blockers, and Historical Notes:** Treat **TRACKER.md** as a living document. It can be organized into sections like “To Do”, “In Progress”, “Done”, and “Blocked” tasks. Each task item might have a brief description and, if useful, a tag or date. For blockers, include a note on what’s needed to unblock (e.g. “Blocked waiting on API keys from third party”). This gives a quick project status overview. Historical notes, such as what was completed in previous sprints or any retrospectives, can either be moved to a **Done/Archive** section in the tracker or recorded in a **HISTORY.md** file. The idea is to not lose significant context: if a feature was postponed or a hack implemented as a temporary fix, note it so the AI doesn’t get confused by seemingly odd code. These historical context notes can also live in the **MANAGER.md** decisions log (e.g. “Temporarily disabled Feature X due to Y, will revisit later”). By capturing tasks and history, an AI assistant could answer questions like “What’s the status of Feature X?” or “Why is module Y not used?” by looking at these notes.

Keeping Context Files Updated Efficiently

Maintaining these files must be an ongoing process – stale documentation can be misleading. Here are some tips to integrate context updates into development without excessive overhead:

- **Adopt a Documentation Routine:** Treat context updates as part of coding. For example, when you implement a new API endpoint, update the relevant **INTERFACE.md** immediately (perhaps even in the same commit). When you fix a bug or make a design change, log the decision or outcome in the **MANAGER.md** or service context. Making this a habit ensures the docs evolve with the code, preventing a large backlog of undocumented changes.
- **Integrate with Development Process:** You can incorporate documentation checks in your workflow. For instance, use PR templates reminding developers to update context files if the change affects docs, or have code reviews include a quick “docs updated?” question. Some teams even fail CI if certain keywords are changed without updating docs (for example, if an API endpoint string is modified, a CI script could flag if INTERFACE.md isn’t updated). While heavy-handed, it reinforces consistency. At minimum, schedule periodic reviews or pair-programming sessions to go over the context files and purge outdated info.
- **Leverage Automation Where Possible:** To reduce manual work, consider semi-automating updates. For example, you could write a simple script to gather all `TODO:` comments or open issues in code and list them in TRACKER.md, or use a documentation generator for parts of the context (like extracting function signatures for an interface document). If you use project management tools (Jira, Trello, etc.), you might sync high-level tasks into TRACKER.md (either manually or via API). Automation can also help generate diagrams from definitions (e.g. using a tool to generate a UML or C4 model diagram from a config). The key is to **automate repetitive parts of documentation** ³ – for example, generating a class diagram or updating a table of contents – so that writing context docs doesn’t become a huge burden.
- **Keep it Lightweight and Regular:** Small, frequent updates are easier than infrequent overhauls. A known pitfall is letting context docs lapse and then spending days to rebuild them later. One

developer described how after “every meaningful change, I’d need to rebuild my context again and again throughout the day” which killed productivity ⁴. Avoid this by maintaining context continuously rather than reconstructing it. Even dedicating 10 minutes at the end of each day or each task to update docs can save hours later on. This continuous approach ensures that both you and your AI assistant always have access to the latest information without a big context-gathering exercise.

- **Use Templates and Consistent Format:** Having a standard format for entries (especially in MANAGER.md or TRACKER.md) can speed up updates. For example, if every decision log entry follows a pattern (Date – Decision – Reason – Status), you won’t spend time figuring out how to write it each time. A documentation framework or template helps maintain consistency ³. This also improves readability (for humans and AI alike). If you foresee certain sections repeating (like for each microservice’s README, you might include a “Tech Stack” subsection), create a template for it. Consistency in structure means updates are just filling in blanks or adding a new bullet, rather than inventing format on the fly.
- **Regularly Prune and Refactor Docs:** Just as code needs refactoring, documentation does too. Periodically clean up the context files: remove or archive information that is no longer relevant. For example, if a feature is long done, you might move its details from the main TRACKER.md to a “Completed” archive section or to a changelog file. If an old decision was reversed, mark it as superseded in the log. Keeping the files lean and relevant prevents context bloat which can confuse AI with outdated info. Aim to keep each context file reasonably sized (e.g. on the order of a few hundred lines at most). In fact, Cursor’s own best practices suggest keeping persistent context snippets under about 500 lines and splitting large topics into multiple files for clarity ⁵. If a file grows too large, consider breaking it into two (for example, if CONTEXT.md becomes huge, maybe split out a separate ARCHITECTURE.md or a **CONTEXT/ directory** with multiple files). Remember, navigability is key – it’s easier to maintain ten focused 100-line files than one monolithic 1000-line file.

Optimizing Context for Cursor (AI Assistance)

Finally, tailor your files for maximum usefulness to AI coding agents like Cursor:

- **Formatting for Clarity:** Use Markdown headings, lists, and tables to make information easy to scan. An AI model will benefit from clear section titles (e.g. “## Database Schema” or “## API Endpoints”) and structured data. Tables can concisely represent relationships (for instance, a table of service interfaces or a comparison of decision options) which is easier for an AI to parse than long paragraphs. Avoid burying critical info in lengthy prose – use bullet points for key facts or requirements so that they stand out ⁶. For example, an AI searching the context might look for the word “endpoint” or a specific function name; if your INTERFACE.md lists them as `**Endpoint:** / users/login - **Purpose:** user authentication`, the model can quickly pick that up.
- **Scope and Relevance:** **Scope each file to a specific domain of context**, as we’ve done with separate files for architecture vs. tasks vs. interfaces. This scoping means when the AI pulls in a file (or you manually reference it with Cursor’s @file command), it’s getting mostly relevant info. Cursor’s context system can automatically include files by relevance; having focused files increases the chance it picks the right context without noise. Also, you can take advantage of Cursor’s project rules

or context directory features: for example, by placing these docs in logical locations or using Cursor's "nested rules" feature, you could auto-attach, say, the API's rules or context when working in the API code ⁷. (In Cursor, rules in subfolders can auto-apply when working in that context.) Consider marking crucial context as "Always include" in Cursor's project rules if available, or simply remember to reference the files with `@filename` in chat when you need that context.

- **File Size Considerations:** As mentioned, keep files reasonably sized and focused. Large Markdown files might not fully fit into the AI's context window, and they may contain sections irrelevant to a specific query. It's better to have multiple smaller files that you can include as needed. For instance, instead of one huge CONTEXT file covering everything, you might have `ARCHITECTURE.md`, `TECH-STACK.md`, `SECURITY_NOTES.md`, etc., linked from CONTEXT or README. This modular approach aligns with recommendations to **split large concepts into multiple composable units** ⁵. The AI (and developers) can then fetch just the relevant piece. If using a context indexing tool, smaller files also update quicker and can be searched more precisely.
- **Auto-Updating and Versioning:** If you implement any automation (like a script that regenerates parts of a context file), ensure it's integrated into your build or deployment process so that it stays up to date. Even without full automation, you might use AI itself to help update context – for example, after implementing a feature, you can prompt Cursor: "Summarize the changes just made in service X and update X's CONTEXT.md accordingly." This human-in-the-loop AI approach can speed up writing documentation. Since all these files are in version control (Git), you also have a history of how context evolved. This versioning is useful: if the AI or team is confused by a recent change, one can quickly check the diff of a context file to see what changed in the understanding of the system.
- **Leverage Context Directories or Specs:** As an advanced tip, you might formalize this structure with a context directory. For example, the *Codebase Context Specification* suggests a `.context/` folder with an `index.md` that provides an architectural overview ¹. You could mirror your root README/CONTEXT info in such an index for tools that specifically look for it. Additionally, adding a note for AI agents (as a comment or in Cursor's rules) to "read the context files before proceeding" can prompt the AI to load up these files. For instance, Cursor or other agents can be given a system instruction like: "If a `.context` directory or context files exist, consult them for project overview and conventions before answering" ⁸. This ensures the AI is primed with the high-level context at the start of a session.
- **Maintain Navigability:** Link related docs to each other. In your root README, include a "**Project Context**" section with links to `CONTEXT.md`, `INTERFACE.md`, etc., so that anyone (or any AI with access to rendered Markdown) can easily jump to them. Within service READMEs, link back to global docs (e.g. "See [Project INTERFACE.md](#) for how this service integrates with others"). These hyperlinks make it straightforward to navigate the knowledge base. An AI might not click links itself, but the existence of the link text (file names, section titles) in multiple places can reinforce the associations (for example, seeing "INTERFACE.md" mentioned in README might clue the AI to check that file for interface-related questions). In Cursor, since it can search the repo, having consistent keywords (like "interface" or "tracker") in file names and headings will help it retrieve the right files when a query is related.

By following these practices, you create a **persistent knowledge hub within your repo** that boosts AI-assisted development. The context files act as the project's "memory," ensuring that important information

is always at hand for both humans and AI. You'll spend less time re-explaining or rediscovering context, and more time building features with the AI's help. In summary, **use a consistent documentation framework, integrate context updates into your workflow, and keep the context focused and up-to-date** ³ ⁹ – this will dramatically improve Cursor's effectiveness as a context-aware coding partner and keep your project running smoothly.

Sources:

- Cursor Documentation – *Project & Nested Rules for Context* ⁷ ⁵
- Codebase Context Specification – *Standard for Context Files* (`.context/index.md`) ¹ ⁸
- Pavel Buchnev, "Context, not prompts" – *On the need to constantly update context documents as code evolves* ⁴ ²
- LinkedIn Discussion – *Best Practices for Maintaining Software Documentation* (use frameworks, automate, and regularly update) ³ ⁹

¹ ⁸ Codebase Context Specification (CCS)

<https://codebasecontext.org/>

² ⁴ Context, not prompts. The missing piece in effective AI-assisted development | by Pavel Buchnev | Medium

<https://butschster.medium.com/context-not-prompts-the-missing-piece-in-effective-ai-assisted-development-080f90174953>

³ ⁹ How to Maintain Documentation for Evolving Software Architecture

<https://www.linkedin.com/advice/3/what-best-strategies-maintaining-documentation-pyodf>

⁵ ⁶ ⁷ Cursor – Rules

<https://docs.cursor.com/context/rules>